

MetaFlow as Scientific Infrastructure for Agricultural AI

A PhD Interview Perspective on Tagliabue et al. (2023) and Tuulos (2022)

Partha Pratim Saha

GreenFieldData MSCA PhD-L
UCBLyon1 / UniMI

Interview, May 2026

Two sources I will discuss today:

Paper: Tagliabue, Tuulos et al., *You Do Not Need a Bigger Boat* (MLSys 2023)

Book: Tuulos, *Effective Data Science Infrastructure* (Manning, 2022)

The Problem: The Dev-to-Production Gap in ML

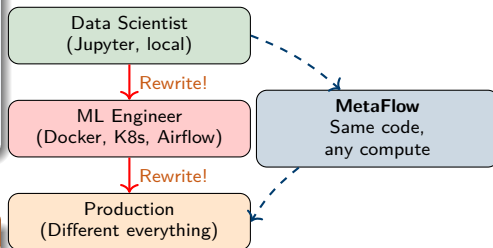
Classic ML Project Failure Pattern

- 1 Notebook prototype works ✓
- 2 Rewrite for production: 6 months
- 3 Rewrite loses reproducibility
- 4 New batch: start over
- 5 **Never shipped, or shipped wrongly**

Root Cause (Tagliabue et al.)

“The main cost is not compute — it is **abstraction mismatch** between data scientists and infrastructure engineers.”

MetaFlow's answer: Prototype and production are *the same code*. The abstraction continuity is not a convenience — it is the scientific contribution.



MetaFlow Philosophy: Abstraction Continuity as a Research Principle

Core Insight (Tuulos, 2022, p. 3)

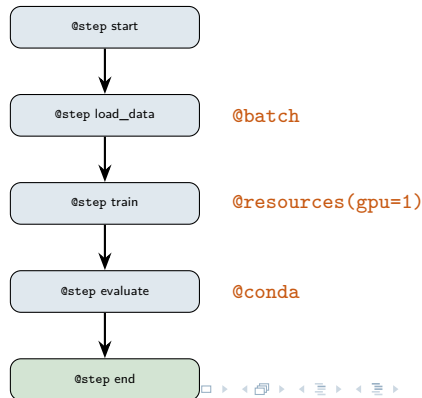
“A data scientist should be able to write code that runs on their laptop and on a cloud cluster without changing a single line of business logic. Infrastructure is a *decorator*, not a rewrite.”

What MetaFlow provides:

- **DAG execution:** steps are Python methods, graph is declared via `self.next()`
- **Artifact versioning:** every step output is a versioned artifact with run ID
- **Compute decorators:** `@batch`, `@kubernetes`, `@conda`, `@resources` — same code, different compute
- **Run lineage:** inspect any historical run, any step, any artifact
- **Resume:** restart from any step after failure

What it deliberately does *not* do:

- Replace data stores (use S3, Postgres, etc.)



V1 — Velocity: Iteration Without Infrastructure Overhead

Problem (Tagliabue et al., 2023)

In ML research, the cost of changing one step should not force re-running all steps. In agricultural AI: re-ingesting 3 years of sensor data just to test a new calibration method wastes research time.

Iterating on calibration step only

```
1 # Run full pipeline first time:
2 python agritalk_flow.py run
3
4 # Saved artifact: run/1234/calibrate/data
5
6 # Iterate on calibration ONLY -- no re-ingestion:
7 python agritalk_flow.py run \
8     --start-step calibrate
9
10 # MetaFlow reuses all upstream artifacts.
11 # Supervisor in Lyon can inspect:
12 from metaflow import Run
13 r = Run("AgriTalkFlow/1234")
14 print(r["calibrate"].task.data.ece_score)
```

Why this matters for PhD research:

- Year 1: iterate on conformal calibration (C1) without re-ingesting sensor corpus
- Year 2: iterate on BVF attribution (C2) without re-running fine-tuning
- Year 3: iterate on federated evaluation without re-training per farm

Key point for interview

Velocity is not about running faster. It is about isolating the research variable you are actually changing. MetaFlow enforces this by design.

Problem in Joint PhD Context

How does a supervisor in Lyon verify that the model I trained and evaluated in Milan used the *correct calibration set*? With a Git repo, they can check code. With MetaFlow, they can inspect the *exact artifact* used at every step.

Cross-site validation (Lyon + Milan)

```
1 from metaflow import Run, namespace
2
3 # Supervisor in Lyon checks Milan run:
4 namespace("partha@unimi")
5 run = Run("AgriTalkFlow/5678")
6
7 # Inspect the exact calibration set:
8 cal_set = run["calibrate"].task.data.cal_set
9 print(f"Calibration size: {len(cal_set)}")
10 print(f"Coverage achieved: "
11       f"{run['calibrate'].task.data.coverage}")
12
13 # Inspect model artifact:
14 model = run["finetune"].task.data.model_path
15 print(f"Model: {model}")
16 print(f"Git SHA: "
17       f"{run.data.git_sha}") # automatic
```

What this enables:

- Dr. Vargas-Solar (Lyon) can validate C1 calibration artifacts remotely
- Prof. Oberti (Milan) can validate C2 attribution runs without GPU access
- Cross-institution paper co-authorship is backed by *verifiable shared artifact state*
- Version mismatches are impossible: artifact ID encodes exact environment

Contrast with Jupyter notebooks:

"It worked on my machine" is a paper validity crisis, not just an inconvenience.

What MetaFlow Versions (Tuulos, 2022, Ch. 4)

Every run receives an immutable run ID. Every artifact (model weights, datasets, evaluation metrics, calibration sets) is content-addressed and retained indefinitely.

Seasonal regression detection

```
1 from metaflow import Flow
2 import pandas as pd
3
4 flow = Flow("AgriTalkFinetuneFlow")
5
6 # Compare all runs across seasons:
7 records = []
8 for run in flow.runs():
9     if run.successful:
10         records.append({
11             "run_id": run.id,
12             "season": run.data.season_tag,
13             "ece": run.data.ece_score,
14             "macro_f1": run.data.macro_f1,
15             "git_sha": run.data.git_sha,
16         })
17
18 df = pd.DataFrame(records)
19 # Detect calibration drift:
20 drift = df.groupby("season")["ece"].mean()
21 print(drift) # Plots seasonal ECE trend
```

PhD-level value of versioning:

- Seasonal replay: *"Would Year 1 calibration have worked on Year 2 data?"* — answerable with evidence
- Model regression detection across research iterations
- Complete provenance chain: corpus → fine-tune → calibrate → evaluate → paper metric
- Every number in every paper can be traced back to an exact run artifact

Interviewer note

This is why MetaFlow is a scientific necessity for

Step Functions and Decorator Semantics

Decorator semantics: same code, different compute

```
1 from metaflow import FlowSpec, step, batch, conda
2
3 class AgriTalkFlow(FlowSpec):
4
5     @step
6     def start(self):
7         """Run locally (lightweight init)"""
8         self.season = "lyon_spring_2027"
9         self.next(self.load_sensor_data)
10
11     @conda(libraries={"kafka-python": "2.0.2"})
12     @step
13     def load_sensor_data(self):
14         """Load from Kafka -- reproducible snapshot"""
15         # Reads from archived Kafka snapshot
16         self.sensor_df = load_kafka_snapshot(
17             topic="agritalk.sensor.raw",
18             timestamp=self.season_start_ts
19         )
20         self.next(self.calibrate)
21
22     @batch(cpu=8, memory=16000) # Cloud GPU
23     @step
24     def calibrate(self):
25         """Conformal calibration -- runs on AWS Batch"""
26         from calibration import conformal_fit
27         self.cal_set, self.coverage = \
28             conformal_fit(self.sensor_df, alpha=0.05)
29         self.next(self.end)
30
31     @step
32     def end(self):
```

Key decorator behaviors:

- @step: marks a DAG node; output is automatically persisted as artifact
- @batch: executes on AWS Batch; same code, no rewrite
- @conda: pins exact dependency versions; reproducibility across machines
- @kubernetes: runs on Kubernetes pod (UCBLyon1 cluster)
- @resources: specifies CPU/GPU/memory per step
- @retry: automatic failure recovery
- @timeout: step-level safety timeout

Key insight: `self.sensor_df` is automatically saved — no pickle, no

DAG as Scientific Object (Not Just a Workflow Tool)

Tagliabue et al. (2023) key argument:

The DAG is not a deployment artifact — it *is* the scientific hypothesis.

In classical ML:

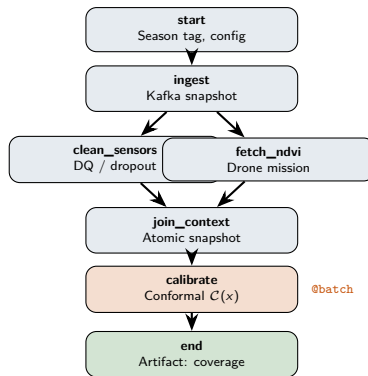
“My experiment” = a notebook + ad-hoc results

In MetaFlow:

“My experiment” = a DAG with versioned artifacts at every step = reproducible scientific object

DAG Card $\equiv$ Model Card (Tagliabue et al.)

The DAG Card (analogous to Model Card) documents: inputs, outputs, each step's purpose, provenance, compute requirements, and test criteria — at the *pipeline* level, not just the model level.



Why MetaFlow is Scientifically Necessary for This PhD

The Non-Stationarity Problem (unique to agricultural AI)

An agricultural AI experiment is not stationary: weather, crop lifecycle, weed pressure, soil variation, pest resistance, and operator vocabulary differ across seasons, farms, and years. A model trained in Lyon spring is evaluated against Milan autumn. **Without explicit versioning of environmental state, comparison is scientifically invalid.**

Velocity in Agriculture

Re-calibrate conformal prediction (C1) for spring wheat without re-ingesting 18 months of sensor data.

Validation in Agriculture

Dr. Vargas-Solar verifies a BVF attribution run (C2) in Lyon using the exact artifact from Milan — not a copy, the same artifact.

Versioning in Agriculture

“Was Year 1 calibration still valid in Year 3?” Seasonal replay answers this with evidence, not assumption.

MetaFlow is contribution C4 — infrastructure that makes C1, C2, C3 reproducibly publishable.
Not a research question; a scientific validity guarantee.

Seasonal Replay: A PhD-Level MetaFlow Feature

Core Question This Enables

"Would the HITL policy calibrated on Lyon winter 2027 data have correctly triggered during UniMI spring 2028 early-season spraying?"

Without MetaFlow: answered with guesswork or a new experiment. With MetaFlow: answered by replaying the archived calibration step against archived sensor snapshots.

Seasonal replay implementation

```
1 from metaflow import Run
2
3 # Load Lyon winter 2027 calibration artifacts:
4 winter_run = Run("AgriTalkFlow/1234")
5 cal_set_winter = winter_run.data.cal_set
6 alpha_winter = winter_run.data.alpha
7
8 # Load Milan spring 2028 sensor snapshot:
9 spring_run = Run("AgriTalkFlow/5678")
10 sensor_spring = spring_run.data.sensor_df
11
12 # Replay calibration against new season:
13 from calibration import conformal_eval
14 coverage_spring = conformal_eval(
15     cal_set=cal_set_winter,
16     test_data=sensor_spring,
17     alpha=alpha_winter
18 )
```

Scientific value:

- Tests whether conformal exchangeability assumption holds across seasons (RQ1 body)
- Provides calibration drift evidence to justify adaptive recalibration mechanism
- Every replay is a traceable experiment with its own run ID
- Requires *zero additional data collection* — uses archived artifacts

Without versioning: "I believe the calibration

MetaFlow vs. Alternatives: Principled Comparison

Tool	Prototype→Prod	Artifact V.	Resume	Compute Decorator	PhD fit
MetaFlow	✓ Same code	✓ Full	✓ Any step	✓ @batch/@k8s	Excellent
Airflow	✗ DAG = deployment	✗ None	~ DAG-level	Plugin only	Scheduling only
Kubeflow	✗ K8s rewrite	~ Limited	~ Limited	✓ Native K8s	Overkill
MLflow	✗ Experiment only	~ Model-level	✗ None	✗ None	No orchestration
W&B	✗ Logging only	~ Run-level	✗ None	✗ None	No pipelines

Why not Airflow?

Airflow schedules computational tasks but does not provide artifact versioning, prototype continuity, or step-level artifact inspection. My research requires all three.

Why not Kubeflow?

Kubeflow provides Kubernetes-native ML pipelines but requires infrastructure expertise beyond a PhD scope. MetaFlow's @kubernetes decorator provides the same compute access with prototype continuity.

Code Walkthrough: AgriTalkFlow in 12 Steps

agritalk_flow.py – simplified structure

```
1 from metaflow import FlowSpec, step, Parameter
2 from metaflow import batch, conda, resources
3
4 class AgriTalkFlow(FlowSpec):
5
6     season = Parameter("season",
7                        default="lyon_spring_2027")
8     alpha = Parameter("alpha",
9                      default=0.05, type=float)
10
11     @step
12     def start(self): self.next(self.ingest)
13
14     @conda(libraries={"kafka-python":"2.0.2"})
15     @step
16     def ingest(self):
17         self.sensor_df = load_kafka(self.season)
18         self.next(self.clean, self.fetch_ndvi)
19
20     @step
21     def clean(self):
22         self.clean_df = dq_check(self.sensor_df)
23         self.next(self.join_context)
24
25     @step
26     def fetch_ndvi(self):
27         self.ndvi = load_ndvi(self.season)
28         self.next(self.join_context)
29
30     @step
31     def join_context(self, inputs):
32         self.context = merge(inputs.clean_df,
```

Pipeline patterns shown:

- **Fan-out:** ingest → clean + fetch_ndvi (parallel)
- **Fan-in:** join_context(inputs) merges fan-out results
- **Parameters:** season and alpha passed from CLI, tracked per run
- **@conda:** pins dependencies per step
- **@batch:** cloud execution for calibration

Running:

```
# Local prototype:
python agritalk_flow.py run
```

Anticipated Interview Questions and My Answers

Q1: Why not just use Jupyter notebooks with Git?

Git versions *code*; MetaFlow versions *artifacts*. Running the same notebook twice with different random seeds or different upstream data produces different results that Git cannot distinguish. MetaFlow's run IDs provide this artifact identity. In a 3-year multi-site PhD, code versioning without artifact versioning is insufficient.

Q2: Is MetaFlow mature enough for research, not just production?

Tagliabue et al. (2023) demonstrate exactly this: researchers at Coveo used MetaFlow to publish reproducible ML papers. The fact that MetaFlow originated at Netflix (Tuulos, 2022) is evidence of production maturity; its adoption in research contexts is the contribution of the paper I am referencing.

Q3: Could you switch to Airflow if MetaFlow were unavailable?

Airflow would handle scheduling and task orchestration, but I would lose step-level artifact versioning and the prototype-to-production continuity. I would likely add MLflow for experiment tracking on top. This is why MetaFlow is the right architectural choice rather than an arbitrary one.

How MetaFlow Structures My 3-Year Research Plan

Year	MetaFlow Pipeline	Key Steps	Seasonal Replay Use
1	AgriTalkIngestFlow	ingest → clean → register	Baseline: Lyon spring 2027
1	AgriTalkFinetuneFlow	fine-tune → eval → calibrate (C1)	Calibration artifact archived
2	AgriTalkStreamFlow	Kafka → context → TSGA (C3)	Lyon artifacts replayed at Milan
2	AgriTalkBVFFlow	attribution → verifier (C2)	Cross-season BVF stability
3	AgriTalkFederatedFlow	per-farm → aggregate → eval	All seasons, all farms
3	AgriTalkHITLStudyFlow	user study data → trust calibration	Replay for counter-factual

Every paper submission includes:

- Run ID(s) for all reported numbers
- Artifact hash for model weights, calibration sets, evaluation metrics
- Seasonal tag: which farm, which season, which sensor configuration
- Reproducibility checklist via DAG Card (Tagliabue et al., 2021)

Research integrity note

Artifact versioning is not optional in agricultural AI. Seasonal variation means “I ran the same code” is insufficient: the data and environment also varied.

Questions I Would Ask My Supervisors About MetaFlow

For Dr. Vargas-Solar (Lyon / Distributed Data)

- 1 How are sensor data snapshots currently archived at UCBLyon1? Is there a standard schema or naming convention I should build the Kafka-to-Metaflow adapter against?
- 2 Is there institutional GPU access (or AWS budget) for @batch steps, or should I architect C1 calibration to run on CPU-only clusters for Year 1?
- 3 Are there existing ProBayes pipelines I should ensure MetaFlow integrates with cleanly (artifact format, schema agreements)?

For Prof. Oberti (Milan / Robotics / Agriculture)

- 1 Which sensors are instrumented on the UniMI experimental farm? Which topics would you recommend as ground-truth sources for NDVI and spray event logging?
- 2 Is there an existing software interface between the farm robots and ROS2 that I should treat as fixed, or is there flexibility in the command interface design for the AgriTalk HITL gate?
- 3 For the Year 2 operator study, how many operators and agronomists can I realistically recruit for the HITL trust calibration experiment?

Thank you. I welcome any questions about the technical or research framing above.